Here's a complete demonstration of creating a custom functional interface and implementing it using lambda expressions in various ways:

---

1. Basic Custom Functional Interface

Step 1: Define the Functional Interface

java

```
@FunctionalInterface
interface Calculator {
    int calculate(int a, int b);  // Single abstract method
}
```

Step 2: Implement using Lambda Expressions

java

```
public class LambdaDemo {
    public static void main(String[] args) {

        // Implementation 1: Addition
        Calculator add = (a, b) -> a + b;
        System.out.println("Addition: " + add.calculate(10, 5)); // 15

        // Implementation 2: Subtraction
        Calculator subtract = (a, b) -> a - b;
        System.out.println("Subtraction: " + subtract.calculate(10, 5)); // 5

        // Implementation 3: Multiplication
        Calculator multiply = (a, b) -> a * b;
        System.out.println("Multiplication: " + multiply.calculate(10, 5)); // 50

        // Implementation 4: Division
        Calculator divide = (a, b) -> {
            if (b == 0) {
                throw new ArithmeticException("Cannot divide by zero");
            }
            return a / b;
```

```
        };
        System.out.println("Division: " + divide.calculate(10, 5)); // 2
    }
}
```

---

2. More Complex Example: String Processor

java

```
@FunctionalInterface
interface StringProcessor {
    String process(String input);
}

public class StringProcessorDemo {
    public static void main(String[] args) {

        // Lambda 1: Convert to uppercase
        StringProcessor toUpper = str -> str.toUpperCase();
        System.out.println(toUpper.process("hello")); // HELLO

        // Lambda 2: Reverse string
        StringProcessor reverse = str -> new StringBuilder(str).reverse().toString();
        System.out.println(reverse.process("hello")); // olleh

        // Lambda 3: Remove whitespace
        StringProcessor trim = str -> str.trim();
        System.out.println(trim.process("  hello world  ")); // hello world

        // Lambda 4: Multiple operations with block
        StringProcessor complex = str -> {
            String result = str.toLowerCase();
            result = result.replace(" ", "_");
            return result;
        };
        System.out.println(complex.process("Hello World")); // hello_world
    }
}
```

---

3. Real-World Example: Employee Filter

java

```
import java.util.*;
import java.util.stream.Collectors;

@FunctionalInterface
interface EmployeeFilter {
    boolean filter(Employee employee);
}

class Employee {
    private String name;
    private int age;
    private double salary;
    private String department;

    public Employee(String name, int age, double salary, String department) {
        this.name = name;
        this.age = age;
        this.salary = salary;
        this.department = department;
    }

    // Getters
    public String getName() { return name; }
    public int getAge() { return age; }
    public double getSalary() { return salary; }
    public String getDepartment() { return department; }

    @Override
    public String toString() {
        return String.format("%s(%d, $%.2f, %s)", name, age, salary, department);
    }
}

public class EmployeeFilterDemo {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
            new Employee("John", 25, 50000, "IT"),
```

```
            new Employee("Jane", 35, 75000, "HR"),
            new Employee("Bob", 28, 60000, "IT"),
            new Employee("Alice", 42, 90000, "Finance"),
            new Employee("Charlie", 30, 55000, "HR")
        );

        // Lambda 1: Filter employees above 30
        EmployeeFilter ageFilter = emp -> emp.getAge() > 30;
        System.out.println("Employees above 30:");
        filterAndPrint(employees, ageFilter);

        // Lambda 2: Filter employees from IT department
        EmployeeFilter deptFilter = emp -> emp.getDepartment().equals("IT");
        System.out.println("\nIT Department employees:");
        filterAndPrint(employees, deptFilter);

        // Lambda 3: Filter employees with salary > 60000
        EmployeeFilter salaryFilter = emp -> emp.getSalary() > 60000;
        System.out.println("\nEmployees with salary > $60,000:");
        filterAndPrint(employees, salaryFilter);

        // Lambda 4: Combine filters (using lambda)
        EmployeeFilter combined = emp ->
            emp.getAge() > 30 && emp.getSalary() > 60000;
        System.out.println("\nEmployees above 30 with salary > $60,000:");
        filterAndPrint(employees, combined);
    }

    public static void filterAndPrint(List<Employee> employees, EmployeeFilter filter) {
        employees.stream()
            .filter(filter::filter)
            .forEach(System.out::println);
    }
}
```

---

4. Using Built-in Functional Interfaces Instead of Custom

You can often use Java's built-in functional interfaces instead of creating custom ones:

java

```
import java.util.function.*;

public class BuiltinLambdaDemo {
    public static void main(String[] args) {

        // Using Predicate<T> instead of custom filter
        Predicate<Integer> isEven = num -> num % 2 == 0;
        System.out.println("Is 4 even? " + isEven.test(4)); // true

        // Using Function<T,R> for transformation
        Function<String, Integer> stringLength = str -> str.length();
        System.out.println("Length of 'Hello': " + stringLength.apply("Hello")); // 5

        // Using Consumer<T> for operations
        Consumer<String> printer = str -> System.out.println("Message: " + str);
        printer.accept("Hello Lambda!"); // Message: Hello Lambda!

        // Using Supplier<T> to generate values
        Supplier<Double> randomValue = () -> Math.random() * 100;
        System.out.println("Random value: " + randomValue.get());

        // Using BiFunction for operations with two inputs
        BiFunction<Integer, Integer, Integer> multiply = (a, b) -> a * b;
        System.out.println("5 * 3 = " + multiply.apply(5, 3)); // 15
    }
}
```

---

5. Lambda as Method Parameter

java

```
@FunctionalInterface
interface Validator {
    boolean validate(String input);
}

public class LambdaParameterDemo {

    // Method that accepts lambda as parameter
    public static void validateInput(String input, Validator validator) {
```

```
        if (validator.validate(input)) {
            System.out.println("✓ Input '" + input + "' is valid");
        } else {
            System.out.println("✗ Input '" + input + "' is invalid");
        }
    }

    public static void main(String[] args) {
        // Pass lambda directly to method
        validateInput("abc123", str -> str.matches("^[a-zA-Z0-9]+$"));

        // Validate email format
        validateInput("test@email.com", str -> str.contains("@") && str.contains("."));

        // Validate minimum length
        validateInput("Short", str -> str.length() >= 8);

        // Store lambda in variable and pass
        Validator notEmpty = str -> str != null && !str.trim().isEmpty();
        validateInput("Hello", notEmpty);
        validateInput("   ", notEmpty);
    }
}
```

Key Points to Remember:

1. @FunctionalInterface annotation is optional but recommended
2. Lambda expression syntax: `(parameters) -> { body }`
3. For single expressions, `{}` and `return` are optional
4. For multiple statements, use `{}` and explicit `return`
5. Lambda can be assigned to the functional interface reference
6. Lambdas can be passed as method arguments
7. Use method references (e.g., `System.out::println`) for cleaner code when applicable

Output of Complete Example:

text

```
Addition: 15
Subtraction: 5
Multiplication: 50
Division: 2
✓ Input 'abc123' is valid
✓ Input 'test@email.com' is valid
✗ Input 'Short' is invalid
✓ Input 'Hello' is valid
✗ Input '   ' is invalid
```